

Lập trình hướng đối tượng

Java cơ bản (1)

GV: ThS. Ngô Tiến Đức

Nội dung chính

- Chương trình Java cơ bản
- Một số quy ước
- Các lỗi thường gặp
- Biến và kiểu dữ liệu
- Các toán tử
- Lớp Scanner



- 3

Chương trình Java cơ bản

- Reserved words/Keywords: Các từ đặc biệt có thể được trình biên dịch tự động hiểu

abstract	default	if	private	this
boolean	do	implements	protected	throw
break	double	import	public	throws
byte	else	instanceof	return	transient
case	extends	int	short	try
catch	final	interface	static	void
char	finally	long	strictfp	volatile
class	float	native	super	while
const	for	new	switch	
continue	goto	package	synchronized	

Chương trình Java cơ bản

- Các chương trình trong Java phải có ít nhất một class
- Các class có thể lồng nhau
- Mỗi class có một tên (định danh) riêng
- Tên class thường bắt đầu bằng chữ in hoa
- Class chính:
 - Phải có tên trùng với tên file
 - Nằm ở ngoài cùng (không nằm trong class nào khác)
 - Được khai báo với từ khóa public (phạm vi truy cập)

Chương trình Java cơ bản

```
// tên file: HelloWorld.java  
class World {  
    ...  
}  
public class HelloWorld { // class chính  
    public static void main(String[] args) {  
        System.out.println("Hello world");  
    }  
}
```

Chương trình Java cơ bản

Quy định về đặt tên trong Java:

- Phân biệt chữ hoa và chữ thường (case-sensitive)
 - Ví dụ: Hello $\neq$ hello $\neq$ HELLO...
- Có thể bao gồm chữ, số và có thể có ký tự đặc biệt '_', '\$', không được bắt đầu bằng số
 - Ví dụ: hello1, ~~1hello~~, hello_world, \$hello,...
- Không được trùng với từ khóa



- ```
public static void main(String[] args) hoặc
public static void main(String args[])
```

- Khi thực thi chương trình sẽ bắt đầu từ phương thức main
- Chương trình có nhiều phương thức main?



# Chương trình Java cơ bản

- Statements (các câu lệnh): Yêu cầu thực hiện một hành động hoặc chuỗi hành động, kết thúc bằng dấu ;

```
System.out.println("Hello world");
```

- Comments (chú thích): Trình biên dịch tự bỏ qua các dòng chú thích

```
// Chú thích trong một dòng
```

```
/*
```

```
Chú thích
```

```
nằm trên nhiều dòng
```

```
*/
```

# Chương trình Java cơ bản

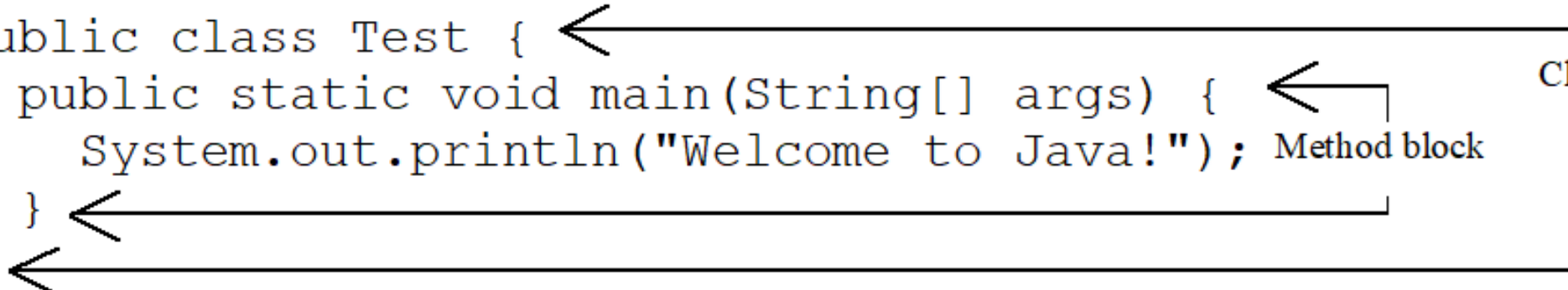
Lệnh in một dòng ra màn hình console:

- `System.out.println("Hello world!")`: in và xuống dòng
- `System.out.print("hello world")`: in và không xuống dòng
- `System.out.println()`: in ra 1 dòng trống (xuống dòng)

# Chương trình Java cơ bản

- Block: Các dòng nằm giữa cặp dấu { }

```
public class Test {
 public static void main(String[] args) {
 System.out.println("Welcome to Java!");
 }
}
```



Class block

Method block

# Một số quy ước

- Đặt tên có ý nghĩa và tránh trùng với các từ khóa
  - ViDu (class)
  - viDu hoặc vi\_du (variable)
  - vidu (package)
  - VIDU hoặc VI\_DU (constant)
  - vi-du (?)
  - vIdU (?)

# Một số quy ước

## Khoảng trắng và lùi dòng

```
// không nên
public class HelloWorld {
 public static void main(String[] args){
 System.out.println ("Hello world");
 }
}
```

```
// nên
public class HelloWorld {
 public static void main(String[] args){
 System.out.println("Hello world");
 }
}
```

# Một số quy ước

Block styles: Nên dùng end-of-line style thay vì next-line style

```
public class HelloWorld {
 public static void main(String[] args){
 System.out.println("Hello world");
 }
} // end-of-line style
```

```
public class HelloWorld
{
 public static void main(String[] args)
 {
 System.out.println("Hello world");
 }
} // next-line style
```

# Các lỗi thường gặp

- Lỗi cú pháp (Syntax Errors): Lỗi trong quá trình biên dịch, có thể được phát hiện bởi trình biên dịch (và IDE). Ví dụ:
  - Sai/thiếu từ khóa: `publish void main...`
  - Đặt tên trùng hoặc không hợp lệ
  - Thiếu dấu ;
  - ...

# Các lỗi thường gặp

- Lỗi thời gian chạy (Runtime Errors): Không phát hiện khi biên dịch nhưng được phát hiện khi thực thi bởi JVM
  - Ví dụ: Dữ liệu không hợp lệ  
`System.out.println(1/0);`
- Lỗi logic (Logic Errors): Chương trình cho ra kết quả không đúng



# Biến và kiểu dữ liệu

- Lưu trữ giá trị của dữ liệu
- Khai báo: <kiểu dữ liệu> <tên biến>;



# Biến và kiểu dữ liệu

- Khởi tạo và gán giá trị cho biến sử dụng ký tự dấu bằng =
- Có thể kết hợp khai báo và khởi tạo, cũng như khai báo đồng thời nhiều biến có cùng kiểu dữ liệu

```
int a = 1, b, c;

b = c = 2;
```

- Khai báo hằng số (constant): sử dụng từ khóa **final**

```
final int A = 1; // giá trị không thể thay đổi
```

# Biến và kiểu dữ liệu

Kiểu dữ liệu nguyên thủy (primitive):

- Số nguyên: **byte, short, int, long**. VD: -1, 0, 69,...
- Số thực: **float, double**. VD: 0.0, 3.14,...
- Ký tự: **char**. VD: '0', 'a', 'A',...
- Giá trị logic: **boolean** (true/false)

# Biến và kiểu dữ liệu

| Kiểu dữ liệu | Kích thước | Giá trị lưu trữ                               |
|--------------|------------|-----------------------------------------------|
| byte         | 1 byte     | Số nguyên từ -128 đến 127                     |
| short        | 2 bytes    | Số nguyên từ -32,768 đến 32,767               |
| int          | 4 bytes    | Số nguyên từ -2,147,483,648 đến 2,147,483,647 |
| long         | 8 bytes    | Số nguyên từ $-2^{63}$ đến $2^{63} - 1$       |
| float        | 4 bytes    | Số thực, đến 7 chữ số phần thập phân          |
| double       | 8 bytes    | Số thực, đến 15 chữ số phần thập phân         |
| boolean      | 1 bit      | true hoặc false                               |
| char         | 2 bytes    | Một ký tự                                     |

# Biến và kiểu dữ liệu

Các kiểu dữ liệu nguyên thủy - số nguyên:

- Mặc định: Hệ cơ số 10
  - VD: `int a = 32;` → in ra 32
- Hệ cơ số 8: Bắt đầu bằng 0
  - VD: `int a = 032;` → in ra...?
- Hệ cơ số 16: Bắt đầu bằng 0x hoặc 0X
  - VD: `int a = 0x1c;` (hoặc 0x1C, 0X1c, 0X1C) → in ra...?

# Biến và kiểu dữ liệu

Các kiểu dữ liệu nguyên thủy - số thực:

- Khai báo biến kiểu **float**: Nên kết thúc bằng ký tự **f** hoặc **F**
  - VD: `float a = 1.1f;`
- Khai báo biến kiểu **double**: Nên kết thúc bằng ký tự **d** hoặc **D**
  - VD: `double a = 1.1d;`
- Khai báo mà không có ký tự kết thúc được hiểu là kiểu **double**



- Có thể biểu diễn dưới dạng số khoa học (scientific number) với ký tự **e** hoặc **E** thay cho lũy thừa của 10
- VD:  $3.14e2 = 314$

# Biến và kiểu dữ liệu

Các kiểu dữ liệu nguyên thủy - số thực:

- Có thể biểu diễn dưới dạng số khoa học (scientific number) với ký tự **e** hoặc **E** thay cho lũy thừa của 10
  - VD:  $3.14e2 = 314$
- Có thể không chính xác?

```
System.out.println(0.1 + 0.2);
```

```
// output: 0.30000000000000004 -> ???
```





- Dạng ký tự: 'a', 'A',...
- Unicode: '\u0061', '\u0041',...
- Mã ASCII: 97, 65,...

- Dạng ký tự: 'a', 'A',...
- Unicode: '\u0061', '\u0041',...
- Mã ASCII: 97, 65,...

# Biến và kiểu dữ liệu

Kiểu dữ liệu không nguyên thủy/tham chiếu (non-primitive/reference):

- Xâu ký tự: **String**. VD: "abc"
- Mảng: **array**. VD: [1,2,3], ['a', 'b'],...
- Lớp: **class**
- Enumeration: **enum**. VD: { SPRING, SUMMER, FALL, WINTER }
- **Interface**

Các kiểu dữ liệu tham chiếu có thể gọi hàm và nhận giá trị *null*

# Biến và kiểu dữ liệu

- Một số ký tự điều khiển nhấn phím:
  - **\b**: xóa 1 ký tự
  - **\n**: dòng mới
  - **\r**: về đầu dòng
  - **\t**: tab
- Hiển thị một số ký tự đặc biệt trong xâu ký tự: thêm **\** trước các dấu nháy kép ( " ), dấu nháy đơn, ( ' ) và dấu backslash ( \ )

# Biến và kiểu dữ liệu

Ép kiểu (casting): Gán giá trị của một kiểu dữ liệu nguyên thủy cho một kiểu khác.

- Ép kiểu mở rộng (widening casting): Ép kiểu dữ liệu từ nhỏ sang kiểu dữ liệu lớn hơn: **byte -> short -> char -> int -> long -> float -> double**
- Có thể thực hiện tự động:

```
int a = 1;
double b = a; // b = 1.0
```

# Biến và kiểu dữ liệu

- Ép kiểu thu hẹp (narrowing casting): Ép kiểu dữ liệu từ lớn sang kiểu dữ liệu nhỏ hơn: **double -> float -> long -> int -> char -> short -> byte**
- Phải thực hiện thủ công bằng cách đặt kiểu dữ liệu trong dấu ( ) phía trước giá trị được gán

```
double a = 1.0d;
int b = (int) a; // b = 1
int c = a; // lỗi
```

# Biến và kiểu dữ liệu

Ép kiểu **char**:

```
char a = '1';
int b = a; // 49
char c = (char) b; // '1'
int d = 666666;
char c = (char) d; // ?
```

# Biến và kiểu dữ liệu

Kết quả của đoạn code này?

```
int n = 2000000000;
System.out.println(n * n);
```

# Biến và kiểu dữ liệu

Hiện tượng tràn (overflow):

```
int n = 2000000000;
System.out.println(n * n);
// output: -1651507200
```

- Kết quả (4,000,000,000,000,000,000) cần biểu diễn bởi 64 bit:

00110111 10000010 11011010 11001110 10011101 10010000 00000000 00000000

↑  
Các bit bậc cao bị loại bỏ

↗  
Các bit bậc thấp được giữ lại

↘  
-1651507200



# Các toán tử

Các toán tử (Operators): Thực hiện các phép biến đổi trên các biến và giá trị

- Toán tử số học (Arithmetic): `+`, `-`, `*`, `/`, `%`, `++`, `--`
- Toán tử bit (Bitwise): `&`, `|`, `^`, `~`, `<<`, `>>`, `>>>`
- Toán tử gán (Assignment): `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `>>=`, `<<=`
- Toán tử so sánh (Comparison): `==`, `!=`, `>`, `<`, `>=`, `<=`
- Toán tử logic (Logical): `&&`, `||`, `!`

# Các toán tử

Toán tử + còn có thể dùng để nối chuỗi ký tự. VD: Các phép toán sau sẽ cho kết quả là gì?

`"hello" + 42`

`1 + "abc" + 2`

`"abc" + 1 + 2`

`1 + 2 + "abc"`

`"abc" + 9 * 3`

`"1" + 1`

`4 - 1 + "abc"`

`"abc" + 4 - 1`

# Lớp Scanner

- Đọc dữ liệu được nhập vào console -> interactive program
- Để sử dụng Scanner cần import từ package **java.util**

```
import java.util.*; // hoặc import java.util.Scanner;
```

- Khởi tạo đối tượng Scanner để đọc từ console:

```
Scanner <name> = new Scanner(System.in);
```

VD: `Scanner sc = new Scanner(System.in);`

Một số phương thức của Scanner:

- `nextInt()`: đọc một giá trị **int**
- `nextDouble()`: đọc một giá trị **double**
- `next()`: đọc một từ (**String**)
- `nextLine()`: đọc một dòng (**String**)
- ...

Các phương thức trên sẽ tạm dừng chương trình cho tới khi người dùng nhập dữ liệu và ấn nút **Enter**

# Lớp Scanner

- Dữ liệu người dùng nhập vào và được đọc bởi Scanner gọi là các **token**
  - Các token được phân cách bởi khoảng trắng (dấu cách, tab, xuống dòng)
- Kiểm tra đầu vào của Scanner còn token nào để đọc hay không: Sử dụng phương thức **boolean** `hasNext()`, `hasNextInt()`, `hasNextDouble()`...
  - Thường áp dụng khi đọc dữ liệu từ file
- Nếu cố tình nhập sai kiểu dữ liệu đầu vào: Chương trình báo lỗi

# Tài liệu tham khảo

- Liang, Y. Daniel, Introduction to Java programming and Data Structures, 11th edition, Pearson Prentice Hall, 2019
- N. M. Sơn, Bài giảng Lập trình hướng đối tượng, HVCNBCVT, 2020
- N. M. Sơn, Slide giảng dạy môn Lập trình hướng đối tượng
- T. T. V. Anh, Slide giảng dạy môn Lập trình hướng đối tượng